NumPy

The Numpy library is popular python library used for scientific computing application. Numpy (Numerical Python or Numeric Python) is an open-source module of python that provides functions for fast mathematical computing on array and matrices.

NumPy arrays are mutable, meaning their contents can be changed after the array is created. You can modify elements, slices, or even entire sections of a NumPy array. This mutability is one of the reasons why NumPy arrays are widely used for numerical computations

Why Mutability is Useful:

- ✓ Efficiency: In-place modifications can be more memory and computationally efficient since they avoid creating copies of the array.
- ✓ Flexibility: You can perform a wide range of operations directly on the array elements, such as arithmetic operations, element-wise comparisons, and more.

NumPy Installation:

Before working with NumPy, it's mandatory that you have to install python and set path as script folder. After than you have to open CMD and type pip install NumPy.

ARRAY:

An array is a container which can hold a fixed number of items and these items should be of same type. It is a contiguous (one after the other).

An array consists of two part: -

- ✓ **Element –** Each item stored in an array is called an element
- ✓ **Index -** Each location of an element in an array has a numerical index start from 0 (zero).

Types of Array

1) One – Dimensional Array (1D Array): It is also known as Vector

0	1	2	3	4	5	6	7	8	9
100	200	300	400	500	600	700	800	900	1000

2) Two – Dimensional Array or ndarray(2D Array): It is also known as Matrices

0	1	2	3	4	5	6	7	8	9
10	15	20	25	30	35	40	45	50	55
60	65	70	75	80	85	90	95	100	105
110	115	120	125	130	135	140	145	150	165
170	175	180	185	190	195	200	205	210	215

Creating One -Dimensional Array:-

```
import numpy as np
mylist=[1,2,3,4,5,6,7,8,9,10]
myarray=np.array(mylist)
print(myarray)
print("==============List Data====================\n",mydata)
```

Creating Multi-Dimensional Array:-

```
import numpy as np
mylist=[[1,2,3,4,5,6],[4,8,7,9,5,1],[4,7,8,1,2,5]]# equals no of elements
myarray=np.array(mylist)
print(myarray)
print("===============nested list============\n",mylist)
```

Data Type in NumPy Array:-

In NumPy, a popular library for numerical computing in Python, the data type of elements in an array is known as dtype. NumPy supports a variety of data types. Following are the some common data types used in NumPy arrays and their shortcuts.

1) **Integer types:**
   - ✓ 'i1' or 'b': 8-bit signed integer (int8)
   - ✓ 'i2' or 'h': 16-bit signed integer (int16)
   - ✓ 'i4' or 'i': 32-bit signed integer (int32)
   - ✓ 'i8' or 'l': 64-bit signed integer (int64)
   - ✓ 'u1': 8-bit unsigned integer (uint8)
   - ✓ 'u2': 16-bit unsigned integer (uint16)
   - ✓ 'u4': 32-bit unsigned integer (uint32)
   - ✓ 'u8': 64-bit unsigned integer (uint64)

2) **Floating-point types:**
   - ✓ f2: Half precision float (float16)
   - ✓ f4 or f: Single precision float (float32)
   - ✓ f8 or d: Double precision float (float64)

3) **Complex number types:**
   - ✓ c8: Complex number represented by two 32-bit floats (complex64)
   - ✓ c16: Complex number represented by two 64-bit floats (complex128)

4) **Boolean type:**
   - ✓ ?: Boolean type (bool)

5) **String types:**
   - ✓ S: Fixed-length ASCII string type (e.g., S10 for a string of length 10)
     U: Fixed-length Unicode string type (e.g., U10 for a Unicode string of length

```
import numpy as np
int_array = np.array([1, 2, 3], dtype='i4') # Integer array using shortcut
float_array = np.array([1.0, 2.0, 3.0], dtype='f8') # Float array using shortcut
complex_array = np.array([1+2j, 3+4j], dtype='c16') # Complex array using shortcut
bool_array = np.array([True, False, True], dtype='?') # Boolean array using shortcut
string_array = np.array(['a', 'b', 'c'], dtype='U1') # String array using shortcut
print("Integer array:", int_array)
print("Float array:", float_array)
print("Complex array:", complex_array)
print("Boolean array:", bool_array)
print("String array:", string_array)
```

Built-In Function to Create NumPy Array:-

1. arange(x) and arrange(x,y,z)#start stop & step
2. zeros(x) and zeros((x,y)) # For 1D & 2D
3. ones(x) and ones((x,y)) # For 1D & 2D
4. linspac(x,y,z)#start stop & no of elements
5. eye(x) #Identity Metrics(rows==columns)
6. copy() method # to copy
7. empty() method# by default initial value
8. full((3,4),10) # Define Specified Value of 3r & 4c

arange() Method

The arange() method creates an array with evenly spaced elements as per the interval.

```
import numpy as np
x1=np.arange(10)#output as 0 to 9
x2=np.arange(1,10)#output as 1 to 9
x3=np.arange(1,100,2)#output as 1 to 100 esc 1
x4=np.arange(1,10,dtype=float)#explicit data type
```

zeros() Method

The zeros() method creates a new array of given shape and type, filled with zeros.

```
import numpy as np
x1=np.zeros(5)#1D Array bydefault Float Datatype
x2=np.zeros((5,5))#2D Array bydefault Float Datatype
x3=np.zeros((5,5,5))#3D Array bydefault Float Datatype
x4=np.zeros(5,dtype=int)#Explicit Datatype
x5=np.zeros((5,5),dtype=int)#Explicit Datatype
```

ones() Method

The ones() method creates a new array of given shape and type, filled with ones.

```
import numpy as np
x1=np.ones(10)#1D Array Bydefault datatype is float
x2=np.ones([10,10])#2D Array Bydefault datatype is float
x3=np.ones((10,10))#2D Array Bydefault datatype is float
x4=np.ones(10, dtype=int)#Explicit Datatype
x5=np.ones((10,10),dtype=int)#Explicit Datatype
x6=np.ones([10,10],dtype=int)#Explicit Datatype
```

linspace() Method

The linspace() method creates an array with evenly spaced elements over an interval.

```
import numpy as np
x1=np.linspace(1,5,10)#10 Elements From 1 To 5 Bydefault Datatype Float
x2=np.linspace(1,5,10, dtype=int)#Explicit Datatype
```

eye() Method

The eye() method creates a 2D array with 1s on the diagonal and 0s elsewhere.

```
import numpy as np
x1=np.eye(5)#2D Array (rows=columns)Bfloat
x2=np.eye(5,dtype=int)#Explicit Datatype
```

copy() method

The copy() method returns an array copy of the given object.

```
import numpy as np
x1=np.eye(5) #Create original Array
x2=np.copy(x1)#Create copy of X1 in X2
```

empty() Method

The empty() method creates a new array of given shape and type, without initializing entries.

```
x1=np.empty(5) # 1D Array
x2=np.empty(5,5)# 2D Array
```

full() Method

The full() method creates a new array of given shape and type, filled with a given fill value.

```
x1=np.empty(2,5) # 1D like [2 2 2 2 2]
x2=np.empty((2,5),8)# 2D Array with 8
```

NumPy Array Indexing & Slicing

Indexing and slicing are fundamental concepts in NumPy that allow you to access and manipulate elements of arrays.

Indexing

Indexing is used to access a single element from a NumPy array. It allows you to retrieve or modify individual elements using their position (index).

Slicing

Slicing is used to access a range of elements from a NumPy array. It allows you to retrieve or modify a subarray (a portion of the array).

1D NumPy Array Indexing & Slicing

```
import numpy as np
x=np.arange(5,100,5)
print(x)#[  5  10  15  20  25  30  35  40  45  50  55  60  65  70  75  80  85  90 95 ]
print(x[5])#Output 30
print(x[0:5])#Output From Index No 0 to 4
print(x[4:8])#Output From Index No 4 to 7
print(x[4:])#Output From Index No 4 to Last
print(x[:4])#Output From Index No 0 to 3
print(x[:])#Output all Elements
print(x[0:10:2])#Output From Index No 0 to 9 skip 1
print(x[::])#Output All elements
print(x[::-1])#From Backword
print(x[-5:])#print last 5 element
```

2D NumPy Array Indexing & Slicing

```
import numpy as np
x=np.arange(100,500,5).reshape(10,8)
print(x,"\n==========OutPut==========")
print(x[5,5])
print(x[5][5])
print(x[2:5])
print(x[2:5,2:5])
print(x[:,:])
```

Note:- We can use looping concept to access single or element from 1D & 2D numpy array.

```
import numpy as np
data=[1,2,3,4,5,6,7,8]
a=np.array(data)
for i in a:
    print(i)

import numpy as np
data=[1,2,3,4,5,6,7,8]
a=np.array(data)
for i in range(len(a)):
    print(a[i])

import numpy as np
data2=[[1,2,3,4,5],[6,7,8,9,10],[11,12,13,14,15]]
b=np.array(data2)
for i in b:
    for j in i:
        print(j)

import numpy as np
data2=[[1,2,3,4,5],[6,7,8,9,10],[11,12,13,14,15]]
b=np.array(data2)
for i in range(len(b)):
    for j in range(len(b[i])):
        print(b[i,j]) # or print(b[i][j])
```

Assignment

1. Write a NumPy program to print the NumPy version in your system.
2. Write a NumPy program to create a 3x3 matrix with values ranging from 2 to 10.
3. Write a NumPy program to create a null vector of size 10 and update sixth value to 11.
4. Write a NumPy program to create an array with values ranging from 25 to 50.
5. Write a NumPy program to reverse an array.
6. Write a NumPy program to create a 2d array with 1 on the border and 0 inside.
7. Write a NumPy program to append values to the end of an array.
8. Write a NumPy program to find the real and imaginary parts of an array of complex numbers.
9. Write a NumPy program to find the number of elements of an array, length of one array element in bytes and total bytes consumed by the elements.
10. Write a NumPy program to test whether each element of a 1-D array is also present in a second array.
11. Write a NumPy program to find common values between two arrays.
12. Write a NumPy program to get the unique elements of an array.
13. Write a NumPy program to find the union of two arrays.
14. Write a NumPy program to create a new array of 3*5, filled with 2.
15. Write a NumPy program to create a 2-D array whose diagonal equals [4, 5, 6, 8] and 0's elsewhere.
16. Write a NumPy program to interchange axes (row into column) of an array.
17. Write a NumPy program to get the number of nonzero elements in an array.
18. Write a NumPy program to replace all elements of NumPy array that are greater than specified array.
19. . Write a NumPy program to remove specific elements in a NumPy array.
20. Write a NumPy program to count the frequency of unique values in NumPy array.

The End